PROJECT TITLE-WEIGHT UNIT CONVERTER

Department of Basic Sciences, CMRIT

Academic Year: 2025–26

CHAPTER 1

INTRODUCTION

Unit conversion is an essential requirement in daily life, science, engineering, and education. Weight measurement units such as kilograms, grams, milligrams, pounds, ounces, and stones are commonly used across different countries and applications. Manual conversion between these units is often time-consuming and prone to calculation errors.
The Weight Unit Converter is a software-based web application developed using HTML, CSS, and JavaScript. The application allows users to convert values between different weight units instantly with high accuracy. It provides a clean, responsive, and user-friendly interface suitable for students and general users.
This project focuses on fundamental concepts of web development, logic implementation, and user interface design.

Brief History of Unit Conversion Systems

Earlier unit conversions were done manually using formulas and tables, which required time and careful calculations. With the advancement of computers, calculator-based and desktop applications were introduced. Modern systems use web-based applications that provide real-time and accurate results with minimal user effort.

Modern Conversion Systems

Modern conversion systems are software-based tools that use predefined conversion factors and algorithms. These systems are fast, reliable, platform-independent, and easy to maintain. Web-based converters are widely used because they do not require installation and can run on any device with a browser.

**CHAPTER 2

ABOUT THE PROJECT**

2.1 Problem Statement

In daily life, academics, and engineering applications, weight conversion between

different units such as kilograms, grams, pounds, ounces, and milligrams is frequently required. Performing these conversions manually is time-consuming and often leads to calculation errors, especially when precision is required. Many available tools are either unreliable, lack multiple unit options, or have a poor user interface. Hence, there is a need for a simple, accurate, and user-friendly software solution to perform weight unit conversions efficiently.

2.2 Objective of the Project

The main objective of this project is to design and develop a software-based Weight Unit Converter that can accurately convert values between different weight measurement units. The project aims to provide instant results with proper precision, reduce manual calculation errors, and offer an easy-to-use interface. Additionally, the project helps in understanding basic concepts of web development using HTML, CSS, and JavaScript.

2.3 Proposed Solution

The proposed solution is a web-based Weight Unit Converter application developed using HTML for structure, CSS for styling, and JavaScript for conversion logic. The application allows users to input a value, select the source unit and destination unit, and obtain the converted result instantly. All conversions are performed by using grams as the base unit to ensure accuracy and consistency. The system works entirely on the client side and does not require an internet connection after loading.

2.4 Advantages of Proposed Solution

The advantages of the proposed solution are as follows:

- Provides fast and accurate weight conversions
- Eliminates manual calculation errors
- User-friendly and responsive interface
- Supports multiple weight units
- Works without external dependencies
- Suitable for educational and practical use

CHAPTER 3

SYSTEM DESIGN

3.1 Design Thinking Process

Empathize:

Understanding user needs such as fast conversion, clarity, and accuracy.
Define:

The main problem identified was the lack of a simple and reliable converter with multiple units.
Ideate:

A web-based solution using HTML, CSS, and JavaScript was chosen.
Prototype:

A working prototype was developed with input fields, unit selection, and result display.
Test:

The system was tested with multiple values and units to ensure accuracy.

3.2 Methodology (Working Procedure)

1. User enters a numeric value
2. User selects the source unit
3. User selects the target unit
4. System converts value using grams as base unit
5. Result is displayed instantly
6. Conversion is stored in history

3.3 System Description

The system is a client-side web application. All computations are performed using JavaScript without any server dependency. The interface is designed using CSS to ensure responsiveness and usability.

CHAPTER 4

IMPLEMENTATION

The application is implemented using the following technologies:

- HTML for structure
- CSS for styling and layout
- JavaScript for logic and interaction

A-HTML SOURCE CODE (index.html)

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Weight Unit Converter</title>
<link rel="stylesheet" href="styles.css">
</head>

<body>
<main class="container">

<header>
<h1>Weight Unit Converter</h1>
<p class="lead">
Convert between kilograms, grams, milligrams, pounds, ounces and stones.
</p>
</header>

<section class="converter" aria-labelledby="converter-heading">
<h2 id="converter-heading">Convert</h2>

<div class="row">
<label for="input-value">Value</label>
<input id="input-value" type="number" inputmode="decimal" placeholder="e.g. 72.5" />
</div>

<div class="row pair">
<div>
<label for="from-unit">From</label>
<select id="from-unit"></select>
</div>

<div>
<label for="to-unit">To</label>
<select id="to-unit"></select>
</div>
</div>

<div class="row controls">
<button id="swap-btn" type="button">  Swap</button>
<button id="convert-btn" type="button">Convert</button>
<button id="clear-btn" type="button" class="ghost">Clear</button>
</div>

<div class="row result">
<label>Result</label>
<div id="output" class="output">—</div>
<button id="copy-btn" type="button">Copy</button>
</div>

<div class="row options">
<label for="precision">Decimal places</label>
<input id="precision" type="number" min="0" max="12" value="4" />
</div>

</section>

```

```
<section class="history">
<h2>History (latest 5)</h2>
<ol id="history-list"></ol>
<button id="clear-history" class="ghost small">Clear history</button>
</section>

<footer>
<p class="muted">Built with — grams used as base unit</p>
</footer>

</main>

<script src="script.js"></script>
</body>
</html>

```

B-JAVASCRIPT SOURCE CODE (script.js)

```
(function () {
 'use strict';

 const FACTORS = {
  kg: 1000,
  g: 1,
  mg: 0.001,
  lb: 453.59237,
  oz: 28.349523125,
  st: 6350.29318
 };

 const UNITS = [
  { val: 'kg', label: 'Kilogram (kg)' },
  { val: 'g', label: 'Gram (g)' },
  { val: 'mg', label: 'Milligram (mg)' },
  { val: 'lb', label: 'Pound (lb)' },
  { val: 'oz', label: 'Ounce (oz)' },
  { val: 'st', label: 'Stone (st)' }
 ];

 const $ = id => document.getElementById(id);

 const inputValue = $('input-value');
 const fromUnit = $('from-unit');
 const toUnit = $('to-unit');
 const convertBtn = $('convert-btn');
 const swapBtn = $('swap-btn');
 const clearBtn = $('clear-btn');
 const output = $('output');
 const copyBtn = $('copy-btn');
 const precisionInput = $('precision');
 const historyList = $('history-list');
 const clearHistoryBtn = $('clear-history');

 function populateSelects() {
  UNITS.forEach(u => {
   const o1 = document.createElement('option');
```

```
    o1.value = u.val;
    o1.textContent = u.label;
    fromUnit.appendChild(o1);

    const o2 = document.createElement('option');
    o2.value = u.val;
    o2.textContent = u.label;
    toUnit.appendChild(o2);
  });
  fromUnit.value = 'kg';
  toUnit.value = 'lb';
}

function convert(value, unitA, unitB) {
  const grams = value * FACTORS[unitA];
  return grams / FACTORS[unitB];
}

function formatNumber(num, decimals) {
  return Number(num).toLocaleString(undefined, {
    maximumFractionDigits: decimals
  });
}

let history = [];
const HISTORY_MAX = 5;

function renderHistory() {
  historyList.innerHTML = '';
  if (history.length === 0) {
    const li = document.createElement('li');
    li.textContent = 'No conversions yet.';
    historyList.appendChild(li);
    return;
  }
  history.forEach(h => {
    const li = document.createElement('li');
    li.textContent = h;
    historyList.appendChild(li);
  });
}

function doConvert() {
  const value = Number(inputValue.value);
  if (!isFinite(value)) {
    output.textContent = 'Invalid number';
    return;
  }

  const decimals = Math.min(
    Math.max(parseInt(precisionInput.value) || 4, 0),
    12
  );

  const result = convert(value, fromUnit.value, toUnit.value);
  const formatted = formatNumber(result, decimals);

  output.textContent = `${formatted} ${toUnit.value}`;
```

```
    history.unshift(`${value} ${fromUnit.value}     ${formatted} ${toUnit.value}`);
    if (history.length > HISTORY_MAX) history.pop();
    renderHistory();
  }

  function doSwap() {
    const temp = fromUnit.value;
    fromUnit.value = toUnit.value;
    toUnit.value = temp;
    if (inputValue.value) doConvert();
  }

  function doClear() {
    inputValue.value = '';
    output.textContent = '—';
  }

  function doCopy() {
    navigator.clipboard.writeText(output.textContent);
    copyBtn.textContent = 'Copied';
    setTimeout(() => copyBtn.textContent = 'Copy', 1000);
  }

  function clearHistory() {
    history = [];
    renderHistory();
  }

  convertBtn.addEventListener('click', doConvert);
  swapBtn.addEventListener('click', doSwap);
  clearBtn.addEventListener('click', doClear);
  copyBtn.addEventListener('click', doCopy);
  clearHistoryBtn.addEventListener('click', clearHistory);

  populateSelects();
  renderHistory();
})();
```

C -CSS SOURCE CODE (styles.css)

```
    :root{
  --bg:#f6f7fb;
  --card:#ffffff;
  --accent:#0f62fe;
  --muted:#6b7280;
  --radius:12px;
  --shadow: 0 6px 20px rgba(15,22,39,0.06);
  --input-border:#e6e9ef;
  font-family: Inter, system-ui, Arial;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  background: var(--bg);
```

```
  display: flex;
  justify-content: center;
  padding: 24px;
}

.container {
  max-width: 760px;
  width: 100%;
}

.converter, .history {
  background: var(--card);
  padding: 18px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  margin-bottom: 18px;
}

.row {
  display: flex;
  flex-direction: column;
  margin-top: 12px;
}

.row.pair {
  flex-direction: row;
  gap: 12px;
}

label {
  font-size: 0.85rem;
  color: var(--muted);
}

input, select {
  padding: 10px;
  border-radius: 8px;
  border: 1px solid var(--input-border);
  min-height: 44px;
}

.controls {
  flex-direction: row;
  gap: 8px;
}

button {
  background: var(--accent);
  color: white;
  border: none;
  border-radius: 10px;
  padding: 10px 12px;
  font-weight: 600;
  cursor: pointer;
}

button.ghost {
  background: transparent;
```

```
    color: var(--accent);
    border: 1px solid var(--input-border);
}

.output {
    background: #f3f4f6;
    padding: 12px;
    border-radius: 8px;
    font-weight: 600;
}

footer {
    text-align: center;
    color: var(--muted);
    font-size: 0.85rem;
}
```

Conversion is performed by converting the input value to grams and then converting grams into the desired unit. This approach ensures accuracy and consistency.

CHAPTER 5

RESULTS AND ANALYSIS

Sample Test Results

Input	From Unit	To Unit	Output
1	Kilogram	Gram	1000
500	Gram	Kilogram	0.5
16	Ounce	Pound	1
10	Pound	Kilogram	4.5359

Observations

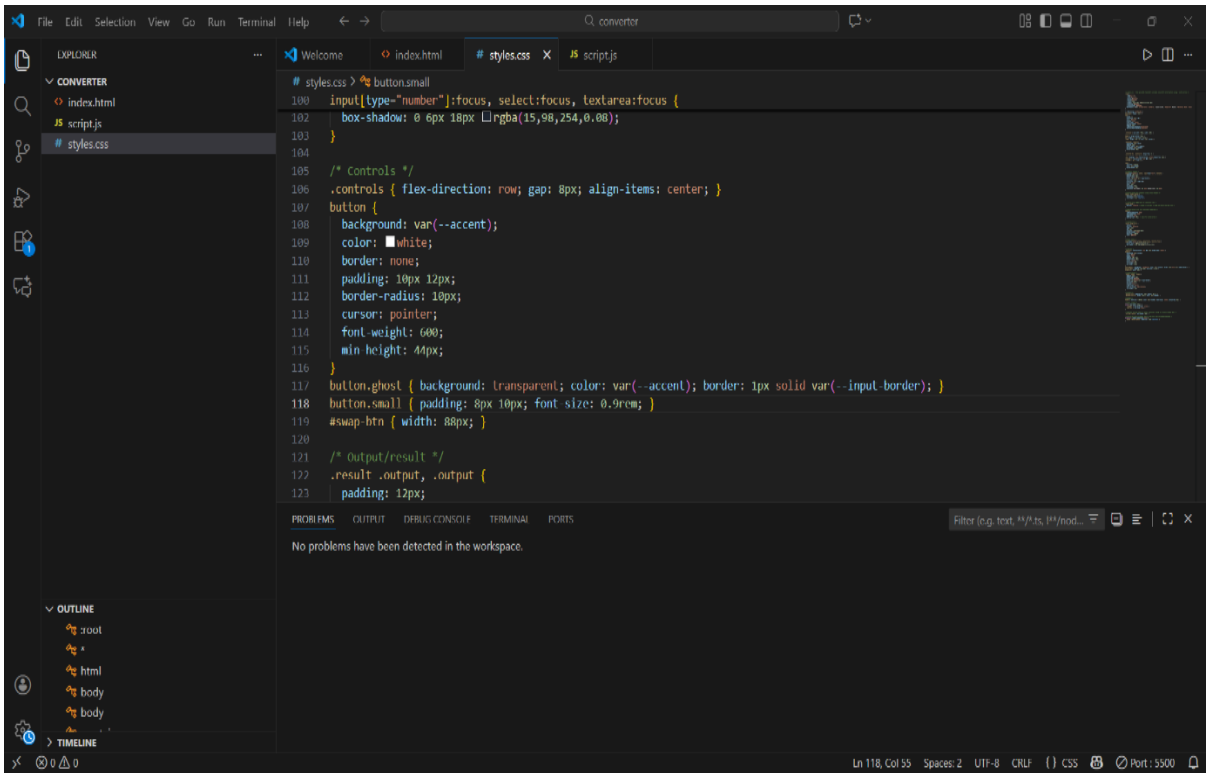
- Conversion is fast and accurate
- User interface is clean and responsive
- History feature improves usability

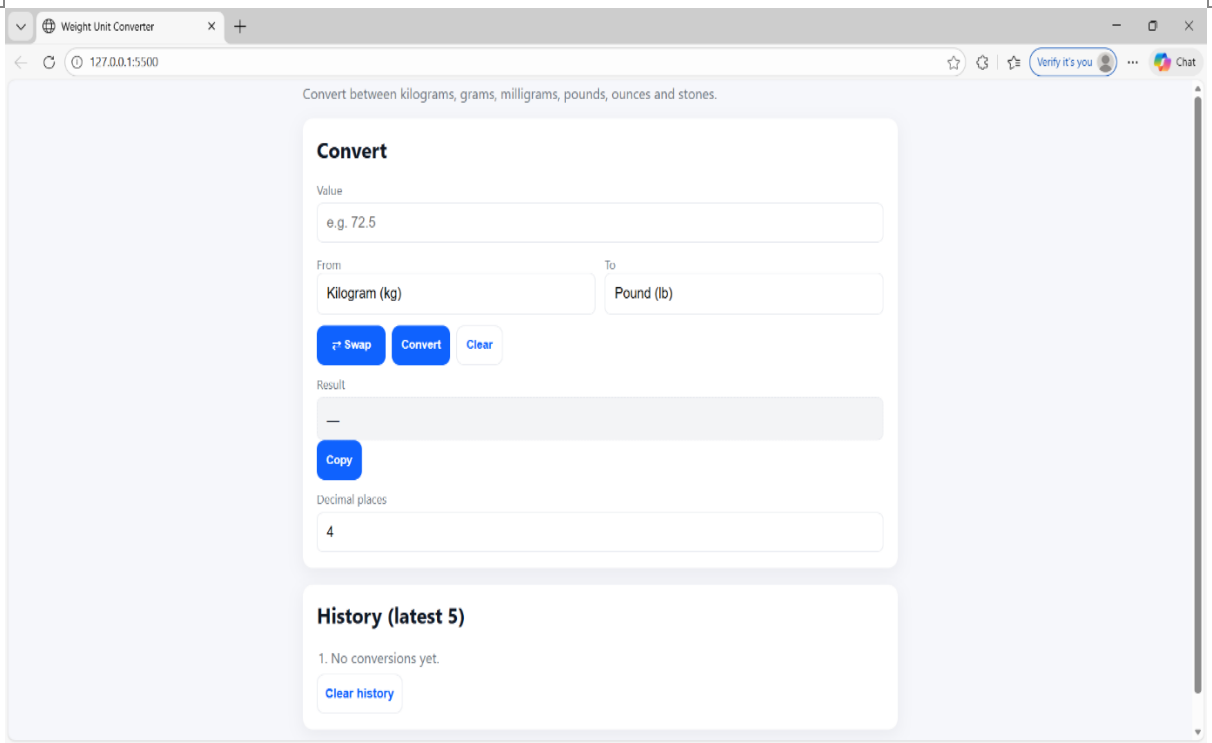
- Decimal precision control enhances accuracy

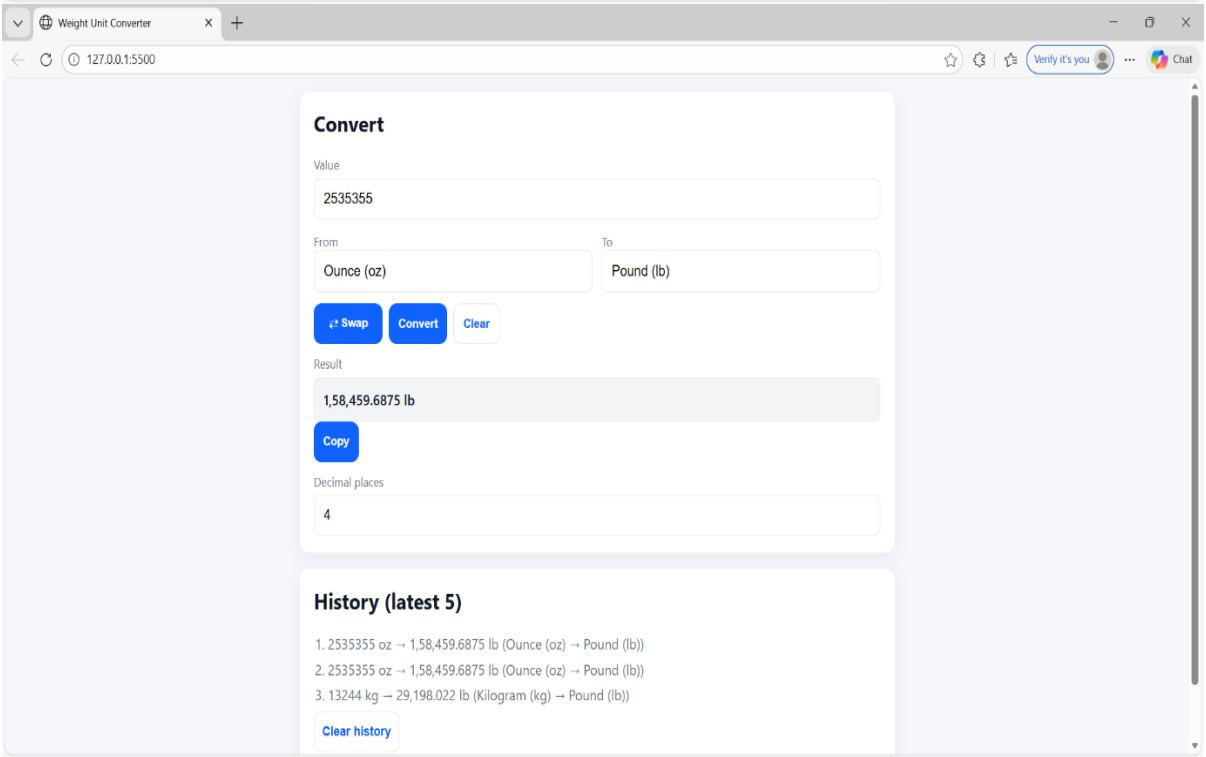
Note

For Software Project, screenshots of the developed website are included. Each screenshot is followed by a brief explanation describing its functionality and output. The results demonstrate that the project objectives have been successfully achieved.

Working prototype screenshots







CHAPTER 6

CONCLUSION & FUTURE WORK

Conclusion

The Weight Unit Converter successfully provides an efficient and accurate solution for converting weight units. The software eliminates manual calculation errors and improves user convenience. It demonstrates effective use of basic web technologies.

Future Work

- Add more units
- Store history using local storage
- Add dark mode
- Mobile application version

REFERENCES

- W3Schools – HTML, CSS, JavaScript
- MDN Web Docs – JavaScript
- Wikipedia – Weight and Mass